ТЩАТЕЛЬНО ПО ПРИЗНАКАМ

Расширенная версия: 27 findings, границы, инварианты и защита перед комиссией

Документ объясняет не только что проверяет признак, но и какая именно граница нарушается, кем она задана и почему это важно.

Что добавлено	Смысл
Границы в каждом признаке	Для каждого finding отдельно расписано: какая граница или инвариант проверяется.
Кто задает границу	Файл на диске, PE/COFF headers, Optional Header, section table, Windows loader, x64 unwind ABI или evidence model проекта.
Почему можно выйти за границу	Повреждение, ручная правка, packer/protector, broken linker metadata, runtime tricks или нестандартная компоновка.
Что должно совпадать	Не raw offset и RVA по числам, а согласованность переводов и ролей: RVA -> section -> file offset/permissions/metadata.

Дата подготовки: 14.07.2026 20:33

1. Главная идея про границы

В 27 признаках слово 'граница' означает не одно и то же. Есть физические границы файла, виртуальные границы образа, границы секций, границы записей RUNTIME_FUNCTION, границы структуры UNWIND_INFO, границы роли секции по permissions и статистические границы evidence model. Поэтому нельзя говорить одной фразой 'границы функций должны совпадать с диском'. Это неверно.

Правильно говорить так: PE имеет две основные координатные системы - raw offsets на диске и RVA в загруженном образе. Они не обязаны иметь одинаковые числовые значения. Они должны быть согласованы через section table. Границы функций задаются в RVA, а для чтения bytes эти RVA переводятся в file offset через VirtualAddress и PointerToRawData соответствующей секции.

Что именно должно быть согласовано

- Raw диапазон секции должен помещаться в фактический размер файла.

- Virtual диапазон секции должен помещаться в SizeOfImage.

- RVA функции должен лежать внутри виртуального образа и обычно попадать в code-like executable section.

- UnwindInfoRVA должен указывать на читаемую UNWIND_INFO metadata внутри image.

- HandlerRVA должен указывать на исполняемый handler target, потому что handler - это код.

- Section permissions должны соответствовать роли данных: metadata обычно не executable и не writable, handler target должен быть executable.

- Процентные пороги 1%, 5%, 10%, 30%, 80% - это не PE-спецификация, а открытая инженерная граница модели подозрительности.

2. Кто устанавливает разные границы

Тип границы	Кем/чем задана	Какие findings используют
Размер файла	Самим файлом на диске	1
SizeOfImage	Optional Header PE, далее интерпретируется загрузчиком Windows	2, 6, 9, 10, 15
Section raw/virtual ranges	Section table: PointerToRawData, SizeOfRawData, VirtualAddress, VirtualSize	1-5, 19-23
Alignment	FileAlignment, SectionAlignment, SizeOfHeaders, SizeOfImage, ImageBase	5
Exception Directory	DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION] в Optional Header	6, 7
RUNTIME_FUNCTION range	Записи .pdata: BeginAddress, EndAddress, UnwindInfoAddress	8-13
UNWIND_INFO layout	Microsoft x64 unwind ABI: header, codes, tail, handlers, chain	14-15
Handler target	EHANDLER/UHANDLER flags и HandlerRVA внутри UNWIND_INFO	18, 22-24, 27
Статистические пороги	Evidence matrix проекта, явно зафиксирована в коде и документации	8-18, 20-21, 26

3. Карта всех 27 признаков

N	Finding	Семейство	Severity	Вес
1	pe_section_raw_oob	table_integrity	high	+25
2	pe_section_virtual_oob	table_integrity	high	+20
3	pe_section_raw_overlap	table_integrity	medium	+15
4	pe_section_virtual_overlap	table_integrity	high	+20
5	pe_section_misaligned	table_integrity	medium	+8
6	pe_exception_directory_oob	table_integrity	high	+25
7	no_runtime_functions	table_integrity	medium	+10
8	invalid_ranges	table_integrity	medium/high	+10 / +20
9	rf_oob	table_integrity	medium/high	+10 / +25
10	unwind_rva_oob	table_integrity	medium/high	+10 / +25
11	rf_unsorted	table_integrity	medium	+15
12	rf_overlaps	table_integrity	high	+25
13	duplicate_begin	table_integrity	medium	+10
14	unwind_parse_fail	unwind_integrity	medium/high	+10 / +25
15	unwind_anomalies	unwind_integrity	medium/high	+10 / +20
16	unwind_reuse_moderate	unwind_reuse	medium	+10
17	unwind_reuse_strong	unwind_reuse	high	+20
18	handler_dominance	handler_dominance	medium	+15
19	unwind_in_exec_section	metadata_placement	high	+20
20	unwind_unusual_section	metadata_placement	medium/high	+8 / +15
21	unwind_writable_section	metadata_placement	medium/high	+10 / +18
22	handler_nonexec_section	metadata_placement	high	+15
23	handler_writable_section	metadata_placement	medium	+10
24	placement_exec_nonexec_handler_combo	metadata_placement	high	+10
25	placement_integrity_combo	unwind_integrity	medium	+8
26	reuse_dominance_combo	unwind_reuse	medium	+8
27	writable_metadata_handler_combo	metadata_placement	high	+10

4. Как читать разбор признака

Дальше каждый finding разбирается как полноценный паспорт инженерного решения. Это сделано специально, чтобы на защите не было темных пятен: по каждому признаку видно, какой объект анализируется, что считается нормой, какая граница или инвариант нарушается, как проект это детектирует, почему это важно, где возможны ложные срабатывания и как вручную подтвердить вывод.

Блок	Что он закрывает
Объект анализа	Какая структура реально проверяется: section header, .pdata, RUNTIME_FUNCTION, UNWIND_INFO, handler target, permissions или статистика.
Нормальное состояние	Как это должно выглядеть в обычном compiler-produced PE32+ AMD64.
Граница/инвариант	Какое правило проверяется: размер файла, SizeOfImage, порядок диапазонов, формат UNWIND_INFO, права секции или порог модели.
Кто задает правило	Файл, PE/COFF, Optional Header, Windows loader, Microsoft x64 unwind ABI или evidence matrix проекта.
Точная логика детекта	Как именно реализация проекта превращает нарушение в finding.
Согласованность	Какие поля должны не совпадать по числам, а логически согласовываться между raw/RVA/sections/metadata.
Причины нарушения	Откуда может появиться проблема: corruption, packer, protector, manual edit, нестандартный compiler/runtime.
Опасность	Как это мешает virtual unwind, stack walking, handler analysis или ручному reverse engineering.
Ложные срабатывания	Какие benign-объяснения возможны и почему finding не является обвинением.
Ручная проверка	Что открыть в PE viewer, Ghidra, hex view или debugger.

5. Разбор каждого признака до основания

Ниже каждый finding разобран не как короткий пункт, а как самостоятельная мини-глава. Признаки - сердце проекта: именно они превращают сырые PE/unwind bytes в объяснимый отчет.

1. pe_section_raw_oob

Поле	Значение
Семейство	table_integrity
Severity	high
Вес	+25

1. Объект анализа

Объект анализа - section header и физическое расположение секции в PE-файле. Проект берет raw_ptr и raw_size из section table, а также file_size из реально загруженного буфера.

2. Нормальное состояние

В нормальном PE каждая секция, у которой SizeOfRawData больше нуля, указывает на существующий диапазон байтов в файле. Секция может иметь virtual tail, которого нет на диске, но заявленный raw range обязан существовать.

3. Какая граница или инвариант проверяется

Физическую raw-границу секции в файле: PointerToRawData + SizeOfRawData не должен выходить за фактический размер файла.

4. Кто устанавливает эту границу или правило

Размер файла задается самим файлом на диске. PointerToRawData и SizeOfRawData задаются section header, который обычно формирует линкер или упаковщик.

5. Точная логика детекта в проекте

В pe_parse для каждой секции считается raw_end = raw_ptr + raw_size. Если raw_size > 0 и raw_end > file_size, счетчик section_raw_oob увеличивается. В heuristics.c этот счетчик превращается в finding pe_section_raw_oob.

6. Что должно быть согласовано

Секция не обязана иметь одинаковые raw offset и RVA. Но raw-диапазон, заявленный section table, должен указывать на реальные байты внутри файла.

7. Как можно нарушить признак

Нарушение появляется, когда header говорит, что секция продолжается дальше конца файла. Такое бывает при обрезке файла, ручной правке заголовков, ошибке упаковщика или намеренной обфускации.

8. Почему это опасно для анализа

Анализатор не может прочитать байты, которых физически нет. Если потом .pdata или .xdata переводятся в этот raw-диапазон, отчет может строиться на несуществующих данных.

9. Конкретный пример

file_size=0x12000, PointerToRawData=0x11000, SizeOfRawData=0x4000. Конец raw-диапазона 0x15000, но файл заканчивается на 0x12000.

10. Почему это finding и почему такая оценка

Это сильный malformed-file signal. Вес +25, потому что нарушена базовая граница чтения файла, а не спорная эвристика.

11. Возможные ложные срабатывания и ограничения

Легитимных объяснений мало. Иногда файл может быть специально обрезан для исследования или не предназначен для запуска, но для обычного PE это сильное нарушение.

12. Как проверить вручную

Открыть section table в PE viewer, взять PointerToRawData и SizeOfRawData проблемной секции, сложить их и сравнить с размером файла на диске.

Формулировка для комиссии

> **Граница установлена не мной: файл имеет конечный размер. Если section header указывает за него, PE layout противоречит физической реальности файла.**

2. pe_section_virtual_oob

Поле	Значение
Семейство	table_integrity
Severity	high
Вес	+20

1. Объект анализа

Объект анализа - виртуальная карта секций. Проект использует VirtualAddress, VirtualSize, SizeOfRawData и SizeOfImage.

2. Нормальное состояние

В нормальном PE виртуальный диапазон секции живет внутри SizeOfImage. Секция может занимать в памяти больше, чем на диске, но все равно не должна выходить за образ.

3. Какая граница или инвариант проверяется

Виртуальную границу секции: VirtualAddress + max(VirtualSize, SizeOfRawData) должен помещаться в SizeOfImage.

4. Кто устанавливает эту границу или правило

SizeOfImage задается Optional Header и используется загрузчиком как размер виртуального образа. VirtualAddress и VirtualSize задаются section table.

5. Точная логика детекта в проекте

Проект берет span = max(VirtualSize, SizeOfRawData), затем virtual_end = VirtualAddress + span. Если span > 0 и virtual_end > SizeOfImage, увеличивается section_virtual_oob.

6. Что должно быть согласовано

Числа raw offset и RVA не должны совпадать. Но виртуальный диапазон секции обязан лежать внутри образа, иначе RVA не относится к загруженному модулю.

7. Как можно нарушить признак

Нарушение бывает, когда section header завышает VirtualSize, VirtualAddress поставлен слишком близко к концу образа, SizeOfImage занижен или заголовки правились вручную.

8. Почему это опасно для анализа

Все ключевые структуры проекта живут в RVA: RUNTIME_FUNCTION, UnwindInfoRVA, HandlerRVA. Если RVA-карта секций выходит за образ, координатная система становится недостоверной.

9. Конкретный пример

SizeOfImage=0x50000, VirtualAddress=0x4F000, span=0x3000. Секция заканчивается на 0x52000, то есть за пределами образа.

10. Почему это finding и почему такая оценка

Вес +20, потому что ломается виртуальная карта PE. Это чуть ниже raw OOB +25, но все равно high.

11. Возможные ложные срабатывания и ограничения

Иногда нестандартные packers могут создавать странный layout, но это все равно означает, что статическая карта image противоречива и требует ручной проверки.

12. Как проверить вручную

Сравнить VirtualAddress + max(VirtualSize, SizeOfRawData) каждой секции с SizeOfImage в Optional Header.

Формулировка для комиссии

Проект проверяет, не претендует ли секция на адреса вне image mapping. Такие RVA нельзя надежно сопоставить с кодом или metadata.

3. pe_section_raw_overlap

Поле	Значение
Семейство	table_integrity
Severity	medium
Вес	+15

1. Объект анализа

Объект анализа - пары физических диапазонов секций в файле.

2. Нормальное состояние

В нормальном PE секции могут идти с padding, но их raw-диапазоны не должны накладываться друг на друга.

3. Какая граница или инвариант проверяется

Пересечение физических raw-диапазонов секций: [PointerToRawData, PointerToRawData + SizeOfRawData).

4. Кто устанавливает эту границу или правило

Raw-диапазоны задаются section headers. Нормальный линкер размещает секции в файле без конфликтов.

5. Точная логика детекта в проекте

Проект для каждой пары секций вызывает ranges_overlap_u64(raw_ptr, raw_size, previous_raw_ptr, previous_raw_size). Если оба диапазона непустые и пересекаются, счетчик raw_overlaps увеличивается.

6. Что должно быть согласовано

Один и тот же байт файла не должен одновременно принадлежать двум секциям. Иначе роль байта зависит от того, какую секцию выбрал инструмент.

7. Как можно нарушить признак

Overlap может появиться из-за ручной упаковки, overlay tricks, повреждения section table или deliberately malformed PE.

8. Почему это опасно для анализа

RVA-to-fileoff может стать неоднозначным. Один tool прочитает байты как .text, другой как .rdata или .xdata. Для unwind metadata это опасно: можно разобрать не те bytes.

9. Конкретный пример

.text raw 0x400..0x8400, .rdata raw 0x8000..0xA000. Область 0x8000..0x8400 имеет двойную секционную роль.

10. Почему это finding и почему такая оценка

Вес +15, потому что raw overlap не всегда ломает loaded image, но создает реальную неоднозначность чтения файла.

11. Возможные ложные срабатывания и ограничения

Некоторые malformed/packed samples намеренно используют overlap. Это не всегда вредоносность, но почти всегда нестандартный layout.

12. Как проверить вручную

Выписать raw start/end всех секций, отсортировать по raw_ptr и проверить, не начинается ли следующая секция до конца предыдущей.

Формулировка для комиссии

Граница здесь - физический интервал секции. Признак нужен, чтобы показать конфликт в представлении файла на диске.

4. pe_section_virtual_overlap

Поле	Значение
Семейство	table_integrity
Severity	high
Вес	+20

1. Объект анализа

Объект анализа - пары виртуальных RVA-диапазонов секций.

2. Нормальное состояние

Один RVA в пределах image должен иметь одну секционную принадлежность и один набор permissions.

3. Какая граница или инвариант проверяется

Пересечение виртуальных RVA-диапазонов секций: [VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData)).

4. Кто устанавливает эту границу или правило

VirtualAddress/VirtualSize задаются section table, а интерпретируются загрузчиком и анализаторами как карта виртуального образа.

5. Точная логика детекта в проекте

Для каждой пары секций проект сравнивает [VirtualAddress, VirtualAddress + max(VirtualSize, SizeOfRawData)). При пересечении увеличивает virtual_overlaps.

6. Что должно быть согласовано

Один RVA должен иметь одну секционную интерпретацию: либо код, либо данные, либо metadata. Две секции на один RVA - конфликт карты.

7. Как можно нарушить признак

Возникает при ручной правке секций, packer tricks, неправильном SizeOfImage/VirtualSize или попытке скрыть данные за пересекающимися headers.

8. Почему это опасно для анализа

RUNTIME_FUNCTION и UNWIND_INFO работают в RVA. Если один RVA попадает в две секции, непонятно, какие права и роль у адреса: executable, writable, xdata, rdata или что-то другое.

9. Конкретный пример

.text RVA 0x1000..0x9000 и .xdata RVA 0x8000..0xA000. RVA 0x8500 принадлежит двум секциям.

10. Почему это finding и почему такая оценка

Вес +20 и high, потому что конфликтует виртуальная координатная система, на которой держится анализ.

11. Возможные ложные срабатывания и ограничения

Упаковщики могут делать намеренно странный layout, но это как раз важно показать аналитику. Для обычного compiler-produced PE это сильная аномалия.

12. Как проверить вручную

Построить виртуальные интервалы секций и проверить, не накладываются ли они по RVA.

Формулировка для комиссии

Это не требование совпадения с диском. Это требование однозначности RVA внутри образа.

5. pe_section_misaligned

Поле	Значение
Семейство	table_integrity
Severity	medium
Вес	+8

1. Объект анализа

Объект анализа - alignment-поля Optional Header и согласованность section table.

2. Нормальное состояние

FileAlignment обычно power-of-two в допустимом диапазоне, SectionAlignment не меньше FileAlignment, VirtualAddress кратен SectionAlignment, raw offsets/sizes кратны FileAlignment.

3. Какая граница или инвариант проверяется

Выравнивание и базовые инварианты: FileAlignment, SectionAlignment, SizeOfImage, SizeOfHeaders, ImageBase, порядок VirtualAddress и выравнивание raw/virtual секций.

4. Кто устанавливает эту границу или правило

Alignment-поля задаются Optional Header, а section headers должны им соответствовать. Эти правила формируются PE-форматом и поведением загрузчика.

5. Точная логика детекта в проекте

pe_parse проверяет file_alignment_valid, section_alignment < file_alignment, compact alignment rules, SizeOfImage % SectionAlignment, SizeOfHeaders % FileAlignment, ImageBase alignment, VirtualAddress/raw_ptr/raw_size alignment и порядок VirtualAddress.

6. Что должно быть согласовано

VirtualAddress должен быть кратен SectionAlignment. Raw pointer/size непустой секции должны быть кратны FileAlignment. SizeOfImage и SizeOfHeaders тоже должны согласоваться с alignment.

7. Как можно нарушить признак

Часто встречается при плохой ручной сборке PE, broken packers, corrupted headers или намеренной нестандартной компоновке.

8. Почему это опасно для анализа

Misalignment не всегда делает файл неисполняемым, но снижает доверие к layout. Если заголовки не соблюдают базовые правила, дальнейшая карта RVA/raw может быть неточной.

9. Конкретный пример

FileAlignment=0x200, а PointerToRawData=0x350. Такой raw offset не лежит на ожидаемой файловой границе.

10. Почему это finding и почему такая оценка

Вес +8, потому что это контекстный санитарный сигнал. Он слабее OOB/overlap, но важен как индикатор качества PE layout.

11. Возможные ложные срабатывания и ограничения

Compact/small alignment возможен у некоторых специальных PE, поэтому это medium и +8, а не high.

12. Как проверить вручную

Проверить Optional Header: SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders, ImageBase; затем section headers на кратность и порядок.

Формулировка для комиссии

Это не главный обвиняющий признак, а ранняя проверка, насколько аккуратно и согласованно сформирован PE.

6. pe_exception_directory_oob

Поле	Значение
Семейство	table_integrity
Severity	high
Вес	+25

1. Объект анализа

Объект анализа - DataDirectory[Exception] в Optional Header и область .pdata.

2. Нормальное состояние

Если Exception Directory присутствует, RVA и Size должны задавать читаемый диапазон внутри SizeOfImage, пригодный для массива RUNTIME_FUNCTION.

3. Какая граница или инвариант проверяется

Границу Exception Directory: ExceptionRVA/ExceptionSize должны быть ненулевыми вместе и диапазон RVA..RVA+Size должен помещаться в SizeOfImage; при чтении таблица должна быть доступна как RUNTIME_FUNCTION entries.

4. Кто устанавливает эту границу или правило

Exception Directory задается DataDirectory в Optional Header. Для x64 он указывает на .pdata, то есть таблицу RUNTIME_FUNCTION.

5. Точная логика детекта в проекте

pe_parse помечает exception_directory_oob, если exc_rva/exc_sz противоречивы или exc_rva + exc_sz > SizeOfImage. Дополнительно чтение runtime functions может вернуть malformed, если directory не читается как entries.

6. Что должно быть согласовано

Data Directory должен указывать на область внутри image, которую можно перевести в file offset и прочитать как массив записей по 12 байт.

7. Как можно нарушить признак

RVA может быть нулевым при ненулевом size, size может быть нулевым при ненулевом RVA, конец может выйти за SizeOfImage, или directory может указывать в область без bytes.

8. Почему это опасно для анализа

Это вход в function table. Если он битый, проект не может надежно получить карту функций и перейти к UNWIND_INFO.

9. Конкретный пример

ExceptionRVA=0x70000 при SizeOfImage=0x60000 или ExceptionSize не кратен 12 байтам.

10. Почему это finding и почему такая оценка

Вес +25, потому что ломается центральная точка входа в .pdata. Без нее анализ unwind невозможен или недостоверен.

11. Возможные ложные срабатывания и ограничения

Если image реально не имеет кода или не нуждается в .pdata, directory может быть пустой. Но OOB/битая directory - другое: она заявлена, но указывает неправильно.

12. Как проверить вручную

В PE viewer открыть Optional Header Data Directories, найти Exception Directory, проверить RVA/Size и перевод RVA в секцию.

Формулировка для комиссии

Граница установлена PE DataDirectory и SizeOfImage. Проект проверяет, существует ли вообще таблица, на которую ссылается заголовок.

7. no_runtime_functions

Поле	Значение
Семейство	table_integrity
Severity	medium
Вес	+10

1. Объект анализа

Объект анализа - наличие исполняемых секций и отсутствие RUNTIME_FUNCTION entries.

2. Нормальное состояние

Для обычного x64 non-leaf кода ожидается наличие .pdata. Пустая .pdata допустима только для ограниченных случаев.

3. Какая граница или инвариант проверяется

Не границу диапазона, а границу ожидания для x64 image: если есть executable section, но Exception Directory пустой и n==0, это требует объяснения.

4. Кто устанавливает эту границу или правило

Ожидание идет от модели Windows x64 unwinding: non-leaf functions обычно имеют RUNTIME_FUNCTION entries.

5. Точная логика детекта в проекте

Finding появляется, когда n == 0, exception_rva == 0, exception_size == 0, и image_has_executable_section(img) возвращает true.

6. Что должно быть согласовано

Если в образе есть реальный исполняемый код, должна существовать карта функций для unwinding, кроме допустимых leaf-only/tiny/resource-only случаев.

7. Как можно нарушить признак

Может быть легитимно для leaf-only stub, tiny executable, forwarding/resource-only image. Может быть подозрительно при удаленной .pdata или ручной сборке.

8. Почему это опасно для анализа

Без function table невозможно восстановить frames для non-leaf функций и обработку исключений обычным x64 механизмом.

9. Конкретный пример

PE имеет .text с executable flag, но ExceptionRVA=0, ExceptionSize=0 и runtime functions count=0.

10. Почему это finding и почему такая оценка

Вес +10 medium: это важный сигнал, но он имеет benign explanations, поэтому не high.

11. Возможные ложные срабатывания и ограничения

Leaf-only tiny executable, forwarding/resource-only image, необычный loader stub. Поэтому finding medium и ослабляется mitigations.

12. Как проверить вручную

Проверить, есть ли реальный код в executable section, есть ли non-leaf functions, есть ли exception directory и почему он пустой.

Формулировка для комиссии

Проект не говорит, что пустая .pdata всегда вредна. Он говорит: при наличии кода это нужно проверить руками.

8. invalid_ranges

Поле	Значение
Семейство	table_integrity
Severity	medium/high
Вес	+10 / +20

1. Объект анализа

Объект анализа - BeginAddress и EndAddress каждой RUNTIME_FUNCTION entry.

2. Нормальное состояние

BeginAddress < EndAddress. Диапазон half-open: начало включительно, конец не включительно.

3. Какая граница или инвариант проверяется

Логическую границу функции в RUNTIME_FUNCTION: EndAddress должен быть строго больше BeginAddress.

4. Кто устанавливает эту границу или правило

BeginAddress/EndAddress задаются каждой RUNTIME_FUNCTION entry. Формат задает half-open range [BeginAddress, EndAddress).

5. Точная логика детекта в проекте

В цикле heuristics.c для каждой записи: если e <= b, увеличивается rf_invalid_ranges и запоминается representative RVA.

6. Что должно быть согласовано

Функция должна иметь непустой прямой диапазон RVA. EndAddress не обязан совпадать с концом raw bytes, но обязан быть после BeginAddress в RVA-пространстве.

7. Как можно нарушить признак

Может появиться при повреждении .pdata, сортировочных ошибках, fake entries, truncation или manual table generation.

8. Почему это опасно для анализа

Если range пустой или обратный, текущий RIP нельзя корректно связать с функцией. Virtual unwind не понимает, какой диапазон покрывает запись.

9. Конкретный пример

BeginAddress=0x2000, EndAddress=0x1F00 или BeginAddress=EndAddress=0x1400.

10. Почему это finding и почему такая оценка

До 1% - +10, выше 1% - +20. Порог отделяет единичную битую запись от системной поломки .pdata.

11. Возможные ложные срабатывания и ограничения

Почти нет нормальных причин для обратного/пустого диапазона, но единичная запись может быть мусором или повреждением, поэтому есть порог 1%.

12. Как проверить вручную

Открыть .pdata, найти entry, сравнить BeginAddress и EndAddress.

Формулировка для комиссии

Граница здесь задана самой записью функции. Функция не может заканчиваться до своего начала.

9. rf_oob

Поле	Значение
Семейство	table_integrity
Severity	medium/high
Вес	+10 / +25

1. Объект анализа

Объект анализа - function range относительно SizeOfImage.

2. Нормальное состояние

BeginAddress и EndAddress должны находиться внутри RVA-пространства образа.

3. Какая граница или инвариант проверяется

Границы BeginAddress/EndAddress относительно SizeOfImage: диапазон функции должен лежать внутри виртуального образа.

4. Кто устанавливает эту границу или правило

RUNTIME_FUNCTION задает Begin/End как RVA, а SizeOfImage задает границу допустимого RVA-пространства image.

5. Точная логика детекта в проекте

Проект проверяет, что Begin/End не выходят за img->size_of_image. При выходе увеличивает rf_oob.

6. Что должно быть согласовано

Границы функций не обязаны совпадать с raw offsets на диске. Они должны быть RVA внутри image и переводиться через section table при чтении bytes.

7. Как можно нарушить признак

EndAddress или BeginAddress могут стать больше SizeOfImage из-за corrupt .pdata, неверной base math, fake entries или obfuscation.

8. Почему это опасно для анализа

Function table начинает описывать код вне модуля. Такой диапазон не может быть надежно найден в секциях и не должен участвовать в normal virtual unwind.

9. Конкретный пример

SizeOfImage=0x50000, BeginAddress=0x70000, EndAddress=0x70100.

10. Почему это finding и почему такая оценка

Одиночные случаи +10, массовые выше 1% +25. Массовый OOB в карте функций - сильное нарушение.

11. Возможные ложные срабатывания и ограничения

Если SizeOfImage сам поврежден, rf_oob может быть следствием PE layout issue. Но это все равно значимо: карта функций не согласована с образом.

12. Как проверить вручную

Сравнить BeginAddress/EndAddress подозрительной записи с SizeOfImage и секцией, куда они должны попадать.

Формулировка для комиссии

SizeOfImage установлен PE-образом. Если RUNTIME_FUNCTION выходит за него, запись описывает функцию вне загруженного модуля.

10. unwind_rva_oob

Поле	Значение
Семейство	table_integrity
Severity	medium/high
Вес	+10 / +25

1. Объект анализа

Объект анализа - UnwindInfoAddress каждой RUNTIME_FUNCTION entry.

2. Нормальное состояние

UnwindInfoRVA должен указывать на существующий UNWIND_INFO внутри image и переводиться в readable bytes.

3. Какая граница или инвариант проверяется

Границу UnwindInfoAddress относительно SizeOfImage: ссылка из RUNTIME_FUNCTION на UNWIND_INFO должна существовать внутри образа.

4. Кто устанавливает эту границу или правило

UnwindInfoAddress задается RUNTIME_FUNCTION. Допустимое RVA-пространство задается SizeOfImage.

5. Точная логика детекта в проекте

Если unwind_rva отсутствует/не читается/выходит за SizeOfImage или pe_rva_to_fileoff не может перевести его к bytes, увеличивается rf_unwind_oob.

6. Что должно быть согласовано

UnwindInfoRVA не обязан лежать рядом с функцией и не обязан быть в .text. Но он обязан указывать на существующую metadata внутри image.

7. Как можно нарушить признак

Может быть нулевым/отсутствующим, больше SizeOfImage, указывать в virtual tail без bytes или в область, которую нельзя прочитать.

8. Почему это опасно для анализа

Связь 'функция -> инструкция по раскрутке кадра' обрывается. Без UNWIND_INFO нельзя восстановить prolog, stack frame, saved registers и handlers.

9. Конкретный пример

Function range валиден, но UnwindInfoAddress=0x900000 при SizeOfImage=0x30000.

10. Почему это finding и почему такая оценка

Выше 1% +25, потому что массовая битость ссылок на UNWIND_INFO фактически ломает unwind-карту.

11. Возможные ложные срабатывания и ограничения

Иногда проблема вызвана битым section layout, а не самой .pdata. Но для результата важно: ссылка function -> unwind metadata нерабочая.

12. Как проверить вручную

Взять UnwindInfoAddress, проверить его относительно SizeOfImage, найти секцию по RVA, перевести в file offset и попробовать прочитать UNWIND_INFO header.

Формулировка для комиссии

> **Это не проверка красоты. Это проверка, существует ли объект, на который ссылается функция.**

11. rf_unsorted

Поле	Значение
Семейство	table_integrity
Severity	medium
Вес	+15

1. Объект анализа

Объект анализа - порядок entries в таблице RUNTIME_FUNCTION.

2. Нормальное состояние

Таблица ожидается отсортированной по BeginAddress для корректного и быстрого поиска функции по RIP.

3. Какая граница или инвариант проверяется

Порядок RUNTIME_FUNCTION entries по BeginAddress: каждая следующая запись должна идти не раньше предыдущей.

4. Кто устанавливает эту границу или правило

Ожидание упорядоченности связано с Windows function table и алгоритмами поиска диапазона по RIP.

5. Точная логика детекта в проекте

Проект проходит исходный массив и считает inversions: текущий BeginAddress меньше предыдущего.

6. Что должно быть согласовано

Сортировка не про raw/virtual совпадение. Это порядок RVA-диапазонов в таблице .pdata.

7. Как можно нарушить признак

Нарушение бывает при ручной вставке entries, merge errors, malformed table или защитной трансформации.

8. Почему это опасно для анализа

Несортированная таблица может ломать binary search assumptions и усложняет ручную верификацию. Даже если записи валидны, карта функций менее надежна.

9. Конкретный пример

BeginAddress идут 0x5000, 0x5200, 0x3000. Третья запись создает inversion.

10. Почему это finding и почему такая оценка

Вес +15: серьезный структурный сигнал, но не такой жесткий, как OOB или overlap.

11. Возможные ложные срабатывания и ограничения

Нестандартный генератор может создать несортированную, но потенциально сортируемую таблицу. Поэтому medium.

12. Как проверить вручную

Выгрузить BeginAddress всех entries и проверить монотонность возрастания.

Формулировка для комиссии

Это проверка порядка карты функций. Карта может содержать валидные точки, но быть организована против ожидаемого поиска.

12. rf_overlaps

Поле	Значение
Семейство	table_integrity
Severity	high
Вес	+25

1. Объект анализа

Объект анализа - пересечения диапазонов функций после сортировки.

2. Нормальное состояние

RUNTIME_FUNCTION ranges не должны перекрывать друг друга. Один RIP - одна function entry.

3. Какая граница или инвариант проверяется

Пересечение виртуальных диапазонов функций [BeginAddress, EndAddress) после сортировки.

4. Кто устанавливает эту границу или правило

Диапазоны задаются RUNTIME_FUNCTION. Модель x64 unwind ожидает, что один RIP покрывается одной функцией.

5. Точная логика детекта в проекте

Проект сортирует пары begin/end, ведет max_end и проверяет: если следующий begin < max_end, есть overlap.

6. Что должно быть согласовано

Функции могут быть соседними, но не должны перекрывать один и тот же RVA-интервал.

7. Как можно нарушить признак

Overlap появляется при corrupt .pdata, duplicate/fake entries, obfuscation, broken linker metadata или ручной правке.

8. Почему это опасно для анализа

Один RIP получает две competing unwind-записи. Непонятно, какой prolog, stack size, saved registers и handler применять.

9. Конкретный пример

A: 0x1000..0x1100, B: 0x1080..0x1200. Диапазон 0x1080..0x1100 принадлежит двум функциям.

10. Почему это finding и почему такая оценка

Вес +25, потому что это прямой конфликт карты функций и virtual unwind.

11. Возможные ложные срабатывания и ограничения

Для нормальной .pdata почти нет. Если это packer/protector, finding все равно полезен как сигнал ручной компоновки.

12. Как проверить вручную

Отсортировать function ranges и проверить, не начинается ли новый range до конца предыдущего максимального range.

Формулировка для комиссии

Граница здесь - диапазон функции. Один адрес инструкции не должен иметь две разные карты раскрутки.

13. duplicate_begin

Поле	Значение
Семейство	table_integrity
Severity	medium
Вес	+10

1. Объект анализа

Объект анализа - повтор одного BeginAddress в нескольких RUNTIME_FUNCTION entries.

2. Нормальное состояние

Одна начальная RVA функции должна иметь одну интерпретацию в function table.

3. Какая граница или инвариант проверяется

Совпадение BeginAddress у разных RUNTIME_FUNCTION entries после сортировки.

4. Кто устанавливает эту границу или правило

BeginAddress задается .pdata entry как начало функции в RVA.

5. Точная логика детекта в проекте

После сортировки проект сравнивает BeginAddress соседних entries. Совпадение увеличивает rf_duplicate_begins.

6. Что должно быть согласовано

Одно начало функции должно вести к одной записи или к однозначной интерпретации. Две записи с одним start конфликтуют.

7. Как можно нарушить признак

Может быть частным случаем overlap, следом ручной вставки, fake entry или генератора, который создал дубли.

8. Почему это опасно для анализа

Аналитик и algorithm lookup не могут однозначно выбрать function record для начала функции.

9. Конкретный пример

Две записи имеют BeginAddress=0x401000, но разные EndAddress и UnwindInfoAddress.

10. Почему это finding и почему такая оценка

Вес +10, потому что duplicate begin важен, но часто слабее общего rf_overlaps.

11. Возможные ложные срабатывания и ограничения

Может быть побочным эффектом overlap или нестандартной генерации, поэтому medium +10.

12. Как проверить вручную

В .pdata найти entries с одинаковым BeginAddress и сравнить EndAddress/UnwindInfoAddress.

Формулировка для комиссии

Это отдельная форма конфликта границы начала функции, удобная для точной диагностики.

14. unwind_parse_fail

Поле	Значение
Семейство	unwind_integrity
Severity	medium/high
Вес	+10 / +25

1. Объект анализа

Объект анализа - бинарный layout UNWIND_INFO по UnwindInfoRVA.

2. Нормальное состояние

Header, CountOfCodes, массив unwind codes, alignment, handler/chain tail должны быть доступны и читаться полностью.

3. Какая граница или инвариант проверяется

Границы чтения структуры UNWIND_INFO: header, CountOfCodes, unwind-code slots, alignment tail, handler или chained RUNTIME_FUNCTION должны физически читаться.

4. Кто устанавливает эту границу или правило

Формат UNWIND_INFO задает Microsoft x64 unwind ABI. CountOfCodes и Flags внутри структуры задают, сколько bytes нужно читать дальше.

5. Точная логика детекта в проекте

unwind_parse возвращает false при unsupported version, truncated header/codes/tail, opcode spanning beyond CountOfCodes, truncated handler RVA/CHAININFO и других невозможных для безопасного чтения случаях.

6. Что должно быть согласовано

Parser должен безопасно пройти от UnwindInfoRVA к bytes структуры. Если CountOfCodes требует больше slots, чем доступно, структура невалидна для анализа.

7. Как можно нарушить признак

Обрезанные bytes, неизвестная version, opcode spans beyond CountOfCodes, truncated CHAININFO, truncated handler RVA, невозможный tail.

8. Почему это опасно для анализа

Если UNWIND_INFO нельзя разобрать, нельзя доверять fixed stack, saved registers, chained record и handler. Virtual unwind из этой функции не имеет надежной инструкции.

9. Конкретный пример

CountOfCodes=20, но в доступной области осталось 8 bytes; flags EHANDLER, но нет 4 bytes HandlerRVA.

10. Почему это finding и почему такая оценка

До 1% +10, выше 1% +25. Массовые parse failures - один из самых сильных признаков поврежденной unwind metadata.

11. Возможные ложные срабатывания и ограничения

Если RVA попадает в virtual tail без raw bytes, parse fail может быть следствием section layout. Но для unwinding результат тот же: metadata не читается.

12. Как проверить вручную

Перейти по UnwindInfoRVA в hex view и вручную проверить header, CountOfCodes, slots и tail по flags.

Формулировка для комиссии

Граница установлена самой структурой UNWIND_INFO. Проект не ругает стиль, он не может безопасно дочитать формат.

15. unwind_anomalies

Поле	Значение
Семейство	unwind_integrity
Severity	medium/high
Вес	+10 / +20

1. Объект анализа

Объект анализа - смысловые поля уже прочитанной UNWIND_INFO.

2. Нормальное состояние

Version допустима, flags совместимы, opcodes известны, operands корректны, CodeOffset внутри prolog, chain не зациклен, handler внутри image.

3. Какая граница или инвариант проверяется

Логические границы внутри уже разобранной UNWIND_INFO: допустимые version/flags/opcodes/operands, CodeOffset внутри prolog, chain sanity, handler OOB.

4. Кто устанавливает эту границу или правило

Смысловые ограничения задаются x64 unwind ABI и логикой virtual unwind. Строгие ABI-only warnings отделены и сами score не повышают.

5. Точная логика детекта в проекте

unwind_parse заполняет bitmask anomalies. В scoring попадают anomalies без strict ABI-only bits: version, truncation, bad opcode/operand, code offset, chain, handler OOB, alignment, operand sanity.

6. Что должно быть согласовано

Структура может читаться, но поля должны иметь смысл: CodeOffset не за пределами prolog, opcode известен, operands не обрезаны, chain не loop, handler внутри image.

7. Как можно нарушить признак

Bad opcode, bad operand, code offset outside prolog, chain loop/depth exceeded, handler >= SizeOfImage, alignment issue, invalid chain.

8. Почему это опасно для анализа

Файл формально дает bytes, но логика unwind может быть недостоверной. Это типично для поврежденной или искусственной metadata.

9. Конкретный пример

PrologSize=5, a unwind code имеет CodeOffset=20; или CHAININFO совмещен с EHANDLER/UHANDLER.

10. Почему это finding и почему такая оценка

Порог high 5%, потому что отдельные anomalies возможны у нестандартных toolchains. Массовость делает сигнал сильным.

11. Возможные ложные срабатывания и ограничения

Некоторые unusual toolchain patterns возможны. Поэтому strict ABI warnings отделены, a high threshold здесь 5%.

12. Как проверить вручную

Посмотреть anomalies в отчете, затем проверить соответствующее поле UNWIND_INFO: flags, opcode, operand, CodeOffset, chain record или HandlerRVA.

Формулировка для комиссии

Это не subjective странность. Это конкретные условия структуры: version, opcode, operands, offsets, chain, handler bounds.

16. unwind_reuse_moderate

Поле	Значение
Семейство	unwind_reuse
Severity	medium
Вес	+10

1. Объект анализа

Объект анализа - распределение UnwindInfoRVA по function table.

2. Нормальное состояние

Некоторый reuse допустим, но один UNWIND_INFO не должен заметно доминировать без объяснения.

3. Какая граница или инвариант проверяется

Статистическую границу reuse: самый частый UnwindInfoRVA занимает больше 10% entries при n > 200 и подтвержден другим static signal.

4. Кто устанавливает эту границу или правило

Порог 10% задает evidence model проекта, а не PE-формат. Он отделяет заметную концентрацию от обычного повторного использования.

5. Точная логика детекта в проекте

Проект считает max_count для каждого UnwindInfoRVA. Moderate candidate: ratio > 0.10 и n > 200. Finding добавляется только при другом static signal и отсутствии strong reuse.

6. Что должно быть согласовано

Reuse UNWIND_INFO не запрещен. Но заметный reuse должен оцениваться только при достаточной выборке и другом подтверждении.

7. Как можно нарушить признак

Возникает при централизованной metadata, protector dispatcher, manual table normalization или большом количестве функций, сведенных к одному unwind blob.

8. Почему это опасно для анализа

Формат может быть валидным, но pattern показывает нетипичную централизацию, которую не поймают OOB/parse checks.

9. Конкретный пример

1000 функций, 130 указывают на один UnwindInfoRVA; ratio=13%, рядом есть placement anomaly.

10. Почему это finding и почему такая оценка

Вес +10, потому что moderate reuse - статистический сигнал и без контекста может быть benign.

11. Возможные ложные срабатывания и ограничения

Много tiny functions с одинаковым prolog могут легитимно использовать похожую metadata. Поэтому требуется corroboration.

12. Как проверить вручную

Найти доминирующий UnwindInfoRVA, посмотреть функции, которые на него ссылаются, и понять, реально ли у них одинаковый frame/prolog.

Формулировка для комиссии

Проект консервативен: moderate reuse не срабатывает на маленьких таблицах и требует corroboration.

17. unwind_reuse_strong

Поле	Значение
Семейство	unwind_reuse
Severity	high
Вес	+20

1. Объект анализа

Объект анализа - сильная концентрация одного UnwindInfoRVA.

2. Нормальное состояние

В большом образе metadata обычно распределена по множеству функций, даже если prolog похожи.

3. Какая граница или инвариант проверяется

Сильную статистическую границу reuse: самый частый UnwindInfoRVA покрывает больше 30% функций при n > 200.

4. Кто устанавливает эту границу или правило

Порог 30% задает evidence matrix как границу 'слишком большая доля для одного unwind blob' в большом образе.

5. Точная логика детекта в проекте

Если max_count / n > 0.30 и n > 200, добавляется unwind_reuse_strong.

6. Что должно быть согласовано

Reuse допустим, но один UNWIND_INFO для трети большого image выглядит архитектурно нетипично.

7. Как можно нарушить признак

Packer/protector может сделать множество функций с одной metadata, fake dispatcher layout или artificial grouping.

8. Почему это опасно для анализа

Сильная централизация снижает доверие к нормальной compiler-produced карте функций и может скрывать реальную структуру frames.

9. Конкретный пример

1200 functions, 430 ссылаются на один UnwindInfoRVA; ratio=35.8%.

10. Почему это finding и почему такая оценка

Вес +20 high: это не прямой OOB, но в большом image такой pattern серьезен.

11. Возможные ложные срабатывания и ограничения

Специальный compiler/runtime может генерировать очень похожие functions, но 30% остается сильным поводом к проверке.

12. Как проверить вручную

Проверить доминирующий UNWIND_INFO, список функций, секции этих функций и наличие других признаков: handler dominance, placement, parse anomalies.

Формулировка для комиссии

Граница 30% не из PE-спецификации. Это открытая инженерная граница для сильного pattern-сигнала.

18. handler_dominance

Поле	Значение
Семейство	handler_dominance
Severity	medium
Вес	+15

1. Объект анализа

Объект анализа - распределение HandlerRVA среди UNWIND_INFO с EHANDLER/UHANDLER.

2. Нормальное состояние

Handlers могут повторяться, но один handler для подавляющего большинства entries требует контекста.

3. Какая граница или инвариант проверяется

Статистическую границу handler reuse: handler_count > 50 и один HandlerRVA покрывает больше 80% handler entries, с фильтрацией benign runtime-like случая.

4. Кто устанавливает эту границу или правило

Порог 80% задает evidence model. HandlerRVA задается UNWIND_INFO tail при EHANDLER/UHANDLER flags.

5. Точная логика детекта в проекте

Если handler_count > 50 и max_handler_count / handler_count > 0.80, проект проверяет static context и runtime-like section filter. Затем добавляет finding.

6. Что должно быть согласовано

Handlers могут повторяться, но подавляющее большинство функций с одним handler target требует объяснения.

7. Как можно нарушить признак

Может быть нормальным runtime handler, C++ personality-like pattern, protector dispatcher или centralized exception logic.

8. Почему это опасно для анализа

Показывает концентрацию логики обработки исключений. Это важно для reverse engineering: один handler может управлять многими frames.

9. Конкретный пример

300 handler entries, 260 указывают на HandlerRVA 0x20000; ratio=86.7%.

10. Почему это finding и почему такая оценка

Вес +15 medium, потому что dominance может быть benign. Проект не делает high без context.

11. Возможные ложные срабатывания и ограничения

C++ runtime или общий personality handler может быть нормальным. Поэтому severity medium и есть фильтрация executable code-like dominant handler без другого контекста.

12. Как проверить вручную

Открыть доминирующий HandlerRVA в disassembler, проверить секцию, код handler, callers/entries и наличие language-specific data.

Формулировка для комиссии

Граница 80% означает подавляющее большинство, а handler_count > 50 защищает от маленьких выборок.

19. unwind_in_exec_section

Поле	Значение
Семейство	metadata_placement
Severity	high
Вес	+20

1. Объект анализа

Объект анализа - секция, куда попадает UnwindInfoRVA.

2. Нормальное состояние

UNWIND_INFO - metadata, обычно read-only data. Она не должна находиться в executable section.

3. Какая граница или инвариант проверяется

Границу роли секции: UNWIND_INFO как metadata не должен находиться в section с IMAGE_SCN_MEM_EXECUTE.

4. Кто устанавливает эту границу или правило

Права секции задаются Characteristics в PE section header, а загрузчик превращает их в memory permissions.

5. Точная логика детекта в проекте

pe_find_section_for_rva(img, unwind_rva), затем проверка IMAGE_SCN_MEM_EXECUTE в section characteristics.

6. Что должно быть согласовано

UNWIND_INFO не является машинным кодом. Ожидаемые места - .xdata, .rdata, .pdata и похожие data sections.

7. Как можно нарушить признак

Packer может смешивать data/code, metadata может быть вручную помещена в .text, или section flags выставлены необычно.

8. Почему это опасно для анализа

Данные unwind оказываются в области, которая объявлена исполняемой. Это мешает доверять роли секции и часто встречается при защитных трансформациях.

9. Конкретный пример

UnwindInfoRVA попадает в .text с execute flag.

10. Почему это finding и почему такая оценка

Вес +20 high: placement сильный, но структура может быть формально читаемой, поэтому не +25.

11. Возможные ложные срабатывания и ограничения

Packer может смешивать code/data. Но именно это полезно отметить для reverse engineer.

12. Как проверить вручную

По UnwindInfoRVA найти section и ее flags. Если есть execute, проверить bytes: это metadata в code-section или section flags намеренно смешаны.

Формулировка для комиссии

Граница здесь смысловая: metadata должна быть данными, а не исполняемым кодом.

20. unwind_unusual_section

Поле	Значение
Семейство	metadata_placement
Severity	medium/high
Вес	+8 / +15

1. Объект анализа

Объект анализа - имя и роль секции для UNWIND_INFO.

2. Нормальное состояние

Ожидаемые имена: .xdata, .rdata, .pdata, .gnu_xdata, .mrdata или секции, содержащие xdata/pdata/rdata.

3. Какая граница или инвариант проверяется

Ожидаемую секционную принадлежность UNWIND_INFO: allowed names .xdata, .rdata, .pdata, .gnu_xdata, .mrdata и похожие xdata/pdata/rdata names.

4. Кто устанавливает эту границу или правило

Whitelist задает проект на основе типичных compiler/linker layouts для unwind metadata.

5. Точная логика детекта в проекте

Проект проверяет allowed/plausible section name. Если секции нет или имя не похоже на unwind/data section, увеличивает unwind_unusual_section.

6. Что должно быть согласовано

UNWIND_INFO может лежать не строго в .xdata, но секция должна выглядеть как data/unwind section, а не произвольная область.

7. Как можно нарушить признак

Custom packer sections, renamed sections, metadata in .packed/.stub/unknown, no section found for RVA.

8. Почему это опасно для анализа

Само по себе имя секции не доказывает проблему, но указывает, что metadata вынесена из привычной зоны и требует ручной проверки.

9. Конкретный пример

UnwindInfoRVA попадает в .zzzz или .packed, где нет xdata/rdata/pdata semantics.

10. Почему это finding и почему такая оценка

До 5% +8, выше 5% +15. Порог мягче, потому что section names разнообразны.

11. Возможные ложные срабатывания и ограничения

Custom toolchain может назвать секцию иначе. Поэтому низкий вес +8 до 5% и placement_only mitigation.

12. Как проверить вручную

Проверить имя секции, ее permissions, neighboring data и почему linker/protector мог поместить unwind metadata туда.

Формулировка для комиссии

> **Проект не требует только .xdata. Он использует расширенный whitelist, поэтому unusual означает действительно нетипичную зону.**

21. unwind_writable_section

Поле	Значение
Семейство	metadata_placement
Severity	medium/high
Вес	+10 / +18

1. Объект анализа

Объект анализа - writable permission секции UNWIND_INFO.

2. Нормальное состояние

Unwind metadata обычно стабильна и read-only. Writable metadata подозрительна как потенциально изменяемая.

3. Какая граница или инвариант проверяется

Границу изменяемости metadata: section, где лежит UNWIND_INFO, не должна иметь IMAGE_SCN_MEM_WRITE.

4. Кто устанавливает эту границу или правило

Writable flag задается section Characteristics. Ожидание read-only metadata идет из обычной модели compiler-produced PE.

5. Точная логика детекта в проекте

После поиска секции для UnwindInfoRVA проверяется IMAGE_SCN_MEM_WRITE.

6. Что должно быть согласовано

UNWIND_INFO описывает frame layout и обычно должен быть стабильным после загрузки. Writable placement означает возможность runtime modification.

7. Как можно нарушить признак

RW/RWX sections, dynamic metadata, protectors, JIT-like mechanisms, careless section permissions.

8. Почему это опасно для анализа

Статический отчет может не совпасть с runtime state, если unwind metadata изменяется. Это риск для анализа исключений и stack walking.

9. Конкретный пример

UnwindInfoRVA попадает в .data или RWX section.

10. Почему это finding и почему такая оценка

До 5% +10, выше 5% +18. Writable сильнее unusual name, но может иметь runtime explanations.

11. Возможные ложные срабатывания и ограничения

JIT-like или dynamic function table mechanisms могут использовать writable memory, но для статического PE это все равно сильный review point.

12. Как проверить вручную

Проверить section flags, runtime table calls и не меняется ли metadata при запуске.

Формулировка для комиссии

Граница здесь - не адресная, а permission boundary: данные unwind потенциально изменяемы.

22. handler_nonexec_section

Поле	Значение
Семейство	metadata_placement
Severity	high
Вес	+15

1. Объект анализа

Объект анализа - секция, куда указывает HandlerRVA.

2. Нормальное состояние

Handler target должен быть исполняемым кодом, потому что обработчику может передаваться управление.

3. Какая граница или инвариант проверяется

Границу исполняемости handler target: HandlerRVA должен попадать в section с IMAGE_SCN_MEM_EXECUTE.

4. Кто устанавливает эту границу или правило

HandlerRVA задается UNWIND_INFO tail при EHANDLER/UHANDLER. Execute permission задается section Characteristics.

5. Точная логика детекта в проекте

Если UNWIND_INFO has_handler, проект ищет section для handler_rva. Если секции нет или нет IMAGE_SCN_MEM_EXECUTE, увеличивает handler_nonexec_section.

6. Что должно быть согласовано

Handler - это код, которому может быть передано управление. Он должен находиться в исполняемой области или хотя бы иметь объяснимый thunk.

7. Как можно нарушить признак

HandlerRVA может быть поврежден, зашифрован/подменяем runtime, указывать в data blob, или section flags могут быть неверны.

8. Почему это опасно для анализа

Exception handler указывает туда, где по правам не должен исполняться код. Это серьезная несогласованность handler layer.

9. Конкретный пример

HandlerRVA попадает в .rdata или .data без execute flag, либо не попадает ни в одну секцию.

10. Почему это finding и почему такая оценка

Severity high, score +15: природа серьезная, но возможны runtime/protector explanations.

11. Возможные ложные срабатывания и ограничения

Некоторые protectors могут расшифровывать/перемещать handler runtime, но статически это все равно серьезная несогласованность.

12. Как проверить вручную

Открыть HandlerRVA в disassembler. Если там data, проверить, не является ли это encoded pointer, thunk, runtime-generated code или ошибка metadata.

Формулировка для комиссии

Обработчик исключения должен быть исполняемым target. Если секция не executable, это надо проверять немедленно.

23. handler_writable_section

Поле	Значение
Семейство	metadata_placement
Severity	medium
Вес	+10

1. Объект анализа

Объект анализа - writable permission секции handler target.

2. Нормальное состояние

Код handler обычно executable/readable, но не writable.

3. Какая граница или инвариант проверяется

Границу изменяемости handler target: section HandlerRVA не должна быть writable в обычном compiler-produced image.

4. Кто устанавливает эту границу или правило

HandlerRVA задается UNWIND_INFO, writable permission - section header Characteristics.

5. Точная логика детекта в проекте

Если section для handler_rva найдена и имеет IMAGE_SCN_MEM_WRITE, добавляется handler_writable_section.

6. Что должно быть согласовано

Handler code обычно read/execute, но не write. Writable target означает потенциально изменяемую логику обработки исключений.

7. Как можно нарушить признак

RWX sections, protectors, self-modifying code, bad linker flags, runtime-generated handlers.

8. Почему это опасно для анализа

Если handler target изменяем, статический анализ может видеть не ту логику, которая будет исполняться.

9. Конкретный пример

HandlerRVA находится в RWX section.

10. Почему это finding и почему такая оценка

Вес +10 medium, потому что writable handler сам по себе может быть особенностью runtime, но в combo становится сильнее.

11. Возможные ложные срабатывания и ограничения

RWX sections часто встречаются у packers/JIT/protectors. Это не verdict, но важный signal.

12. Как проверить вручную

Проверить, является ли секция RWX, изменяется ли handler bytes runtime, и есть ли dynamic function table activity.

Формулировка для комиссии

Это не утверждение 'точно вредно'. Это предупреждение: логика handler потенциально изменяема.

24. placement_exec_nonexec_handler_combo

Поле	Значение
Семейство	metadata_placement
Severity	high
Вес	+10

1. Объект анализа

Объект анализа - совместное нарушение ролей metadata и handler.

2. Нормальное состояние

Metadata in data-like section, handler in executable section.

3. Какая граница или инвариант проверяется

Сочетание двух boundary violations: UNWIND_INFO в executable section и HandlerRVA в non-executable/unknown section.

4. Кто устанавливает эту границу или правило

Обе границы задаются section permissions: metadata не должна выглядеть как code, handler должен быть code.

5. Точная логика детекта в проекте

Если unwind_exec_section > 0 и handler_nonexec_section > 0, heuristics добавляет combo finding.

6. Что должно быть согласовано

Нормальная картина: metadata в data-like section, handler in executable code section. Здесь роли перевернуты.

7. Как можно нарушить признак

Packer/protector, manual tampering, broken flags, obfuscation that hides code/data roles.

8. Почему это опасно для анализа

Одна ошибка placement может быть объяснима. Две противоположные ошибки в metadata/handler layer вместе намного труднее объяснить benign-компоновкой.

9. Конкретный пример

UNWIND_INFO лежит в .text, а HandlerRVA указывает в .rdata.

10. Почему это finding и почему такая оценка

Добавочный +10: первичные признаки уже дали score, combo усиливает связанность evidence.

11. Возможные ложные срабатывания и ограничения

Может быть broken section flags, но именно сочетание делает случай сильнее.

12. Как проверить вручную

Проверить оба representative RVAs: UNWIND_INFO section и HandlerRVA section. Объяснить, почему роли code/data поменялись местами.

Формулировка для комиссии

Combo показывает не новый формат, а опасное совпадение двух нарушенных границ.

25. placement_integrity_combo

Поле	Значение
Семейство	unwind_integrity
Severity	medium
Вес	+8

1. Объект анализа

Объект анализа - связь placement anomaly с structural integrity anomaly.

2. Нормальное состояние

Даже необычное placement может быть читаемым. Если одновременно есть parse/OOB/anomaly, это сильнее.

3. Какая граница или инвариант проверяется

Связку placement boundary issue с integrity boundary issue: executable/nonexec placement плюс parse/anomaly/unwind_rva_oob.

4. Кто устанавливает эту границу или правило

Placement задается section permissions/names, integrity - форматными границами RUNTIME_FUNCTION/UNWIND_INFO.

5. Точная логика детекта в проекте

Combo добавляется, если есть unwind_exec_section или handler_nonexec_section, и одновременно unwind_parse_fail, unwind_anomalies или rf_unwind_oob.

6. Что должно быть согласовано

Если metadata странно лежит, но идеально читается, это одно. Если странно лежит и еще повреждена по структуре, evidence сильнее.

7. Как можно нарушить признак

Часто возникает при obfuscation/protection, где metadata не только переносится, но и становится частично невалидной.

8. Почему это опасно для анализа

Сигналы из разных слоев подтверждают друг друга: место подозрительное и содержимое тоже проблемное.

9. Конкретный пример

UNWIND_INFO в executable section и одновременно unwind_parse_fail по части записей.

10. Почему это finding и почему такая оценка

Вес +8, потому что это context booster, а не самостоятельная первичная граница.

11. Возможные ложные срабатывания и ограничения

Зависит от истинности первичных findings. Если один первичный false positive, combo ослабевает.

12. Как проверить вручную

Проверить primary placement finding и primary integrity finding, затем решить, связаны ли они одной причиной.

Формулировка для комиссии

Этот признак показывает корреляцию: не просто 'лежит необычно', а 'лежит необычно и плохо читается'.

26. reuse_dominance_combo

Поле	Значение
Семейство	unwind_reuse
Severity	medium
Вес	+8

1. Объект анализа

Объект анализа - связь metadata reuse и handler dominance.

2. Нормальное состояние

Reuse и общий handler допустимы по отдельности, но вместе выглядят как centralized dispatcher pattern.

3. Какая граница или инвариант проверяется

Связку двух статистических границ: UnwindInfoRVA reuse >10% и handler dominance.

4. Кто устанавливает эту границу или правило

Обе границы заданы evidence model: 10% для заметного metadata reuse и 80%/handler_count>50 для dominance.

5. Точная логика детекта в проекте

Если unwind_reuse_ratio > 0.10 и уже есть handler_dominance, добавляется reuse_dominance_combo.

6. Что должно быть согласовано

Reuse metadata и dominance handler по отдельности могут быть benign. Вместе они похожи на centralized dispatcher design.

7. Как можно нарушить признак

Protector/runtime может свести множество функций к одному unwind blob и одному handler target.

8. Почему это опасно для анализа

Формат может быть корректным, но архитектурный pattern говорит о централизованной логике обработки.

9. Конкретный пример

15% функций используют один UNWIND_INFO, а 90% handler entries ведут к одному HandlerRVA.

10. Почему это finding и почему такая оценка

Вес +8, потому что это статистическая корреляция, не format violation.

11. Возможные ложные срабатывания и ограничения

Runtime/library personality handler может быть нормальным. Поэтому combo medium +8.

12. Как проверить вручную

Проверить доминирующий UNWIND_INFO и доминирующий HandlerRVA: действительно ли они образуют общий dispatcher.

Формулировка для комиссии

> **Проект видит не только битые поля, но и suspicious shape всей таблицы.**

27. writable_metadata_handler_combo

Поле	Значение
Семейство	metadata_placement
Severity	high
Вес	+10

1. Объект анализа

Объект анализа - совместная изменяемость unwind metadata и handler target.

2. Нормальное состояние

Unwind metadata стабильна, handler code не writable. Одновременная writable-зона для обоих слоев необычна.

3. Какая граница или инвариант проверяется

Сочетание двух writable boundaries: writable UNWIND_INFO и writable HandlerRVA target.

4. Кто устанавливает эту границу или правило

Writable flags задаются section Characteristics. Metadata и handler target приходят из RUNTIME_FUNCTION/UNWIND_INFO.

5. Точная логика детекта в проекте

Если unwind_writable_section > 0 и handler_writable_section > 0, добавляется combo.

6. Что должно быть согласовано

Нормально, когда unwind metadata стабильна, а handler code не writable. Здесь потенциально изменяемы обе стороны exception mechanism.

7. Как можно нарушить признак

RWX packer sections, runtime-generated code/metadata, self-modifying handler logic, careless permissions.

8. Почему это опасно для анализа

Статическая картина может быть изменена во время выполнения и по карте стека, и по логике handler.

9. Конкретный пример

UNWIND_INFO в .data и HandlerRVA тоже в writable section.

10. Почему это finding и почему такая оценка

Добавочный +10 high: вместе эти два writable признака намного сильнее, чем каждый отдельно.

11. Возможные ложные срабатывания и ограничения

JIT/protector может использовать writable/executable memory, но для обычного PE это сильный review point.

12. Как проверить вручную

Проверить permissions обеих секций, runtime modifications, dynamic function table calls и соответствие static/dynamic report.

Формулировка для комиссии

Это один из самых понятных combo: изменяемая карта unwind плюс изменяемый обработчик исключений.

6. Как отвечать на главный вопрос про границы

Если комиссия спросит: 'какие именно границы нельзя нарушать и почему', нельзя отвечать только 'границы функций'. Нужно назвать слои. Сначала есть физическая граница файла. Потом виртуальная граница SizeOfImage. Потом границы секций. Потом границы записей RUNTIME_FUNCTION. Потом границы структуры UNWIND_INFO. Потом границы роли секции: data, code, writable, executable. И отдельно есть инженерные статистические пороги, которые не являются PE-правилом, а нужны для оценки массовости.

Границы функций задаются не на диске, а в RVA. Они не должны совпадать численно с file offsets. Они должны быть валидны в виртуальном образе и переводимы через section table. Если Begin/End или UnwindInfoRVA выходят за SizeOfImage, это не 'несовпадение с диском', а указание на адрес вне загруженного image.

Финальная короткая формула

Проект проверяет не совпадение raw и RVA по числам, а согласованность всей цепочки: PE headers -> section table -> RVA-space -> .pdata -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler -> section permissions -> итоговая evidence model.

Конец документа.